

Behavioral Cloning

Team ID: SDCC2021001

Presenter: Anawil Tiwari

PROJECT - 3 REPORT

Submitted by: Team Avida of *National Institute of Technology Agartala*

For: SAE Self Driving Car Challenge 2022

Behavioral Cloning

PROJECT REPORT

For SAE India (Southern Section) SDCC

Submitted by,

Team AVIDA (NIT Agartala)

in partial fulfillment for the completion of

SAE Self-Driving Car Challenge

Under the guidance of,

Mr. Shanmugam Sivaraman

(Project Mentor)

DECLARATION

I the undersigned solemnly declare that the project report titled LANE FINDING BASIC is based on my own work carried out during the course of our study under the supervision of Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section.

I assert the statements made and conclusions drawn are an outcome of my research work. I further certify that,

I. The work contained in the report is original and has been done by me under the general supervision of my supervisor.
II. The work has not been submitted to any other institution for any other degree/diploma/certificate in this university or any other university of India or abroad.
III. I have followed the guidelines provided by the university in writing the report.
IV. Whenever I have used materials (data, theoretical analysis and text) from other sources, I have given due credit to them in the references.

KeerthanaNeigapula
Hareesh M
Shivansh Mishra
Dikshita Jaraut
Sowjanya Matakala
Sumit Anand
Anawil Tiwari
Krushnakant Borude
Thanesh Bisen
Rakshit Gupta

Date – 09/06/2022

ACKNOWLEDGEMENTS

I would like to use this opportunity to thank my mentor Mr. Shanmugam Sivaraman, Secretary, SAE India Southern section. for guiding me throughout the project duration. I would also like to thank SAE India Southern Section for providing me an opportunity to work on this interesting project.

I owe gratitude to various books and online resources for the completion of this project. These resources have been mentioned in the references.

Project # 3

Behavioral Cloning:

Introduction:

In this project, we design, train, and test a convolutional neural network (CNN) to clone the driving behaviour from sample images recorded from Udacity's driving simulator.
Behavioral cloning is a method by which human sub-cognitive skills can be captured and reproduced in a computer program. As the human subject performs the skill, his or her actions are recorded along with the situation that gave rise to the action. A log of these records is used as input to a learning program.
The goals / steps of this project are the following: Use the simulator to collect data of good driving behavior Build, a convolution neural network (using Keras) that predicts steering angles from images Train and validate the model with a training and validation set Test that the model successfully drives around the track without leaving the road

Quickstart:

- Two driving modes:

Training: For user to take control over the car Autonomous: For car to drive by itself

- Collecting data: User drives on track 1 and collects data by recording the driving experience by toggle ON/OFF the recorder. Data is saved as frame images and a driving log which shows the location of the images, steering angle, throttle, speed, etc. Another option is trying on Udacity data sample.

Approach:

To have any idea to start this project, the CNN that was eventually used was based on NVIDIA's End to End Learning for Self-Driving Cars paper with different input image size and with dropout added to improve robustness. From the paper, data collection is the first important part. Per project requirement, data collection can only performed on Track 1. We drove about 4 laps around Track 1 by keyboard control to collect data. The driving wasn't extrememly smooth as actual driving. So we decided to use Udacity sample data as starting point.

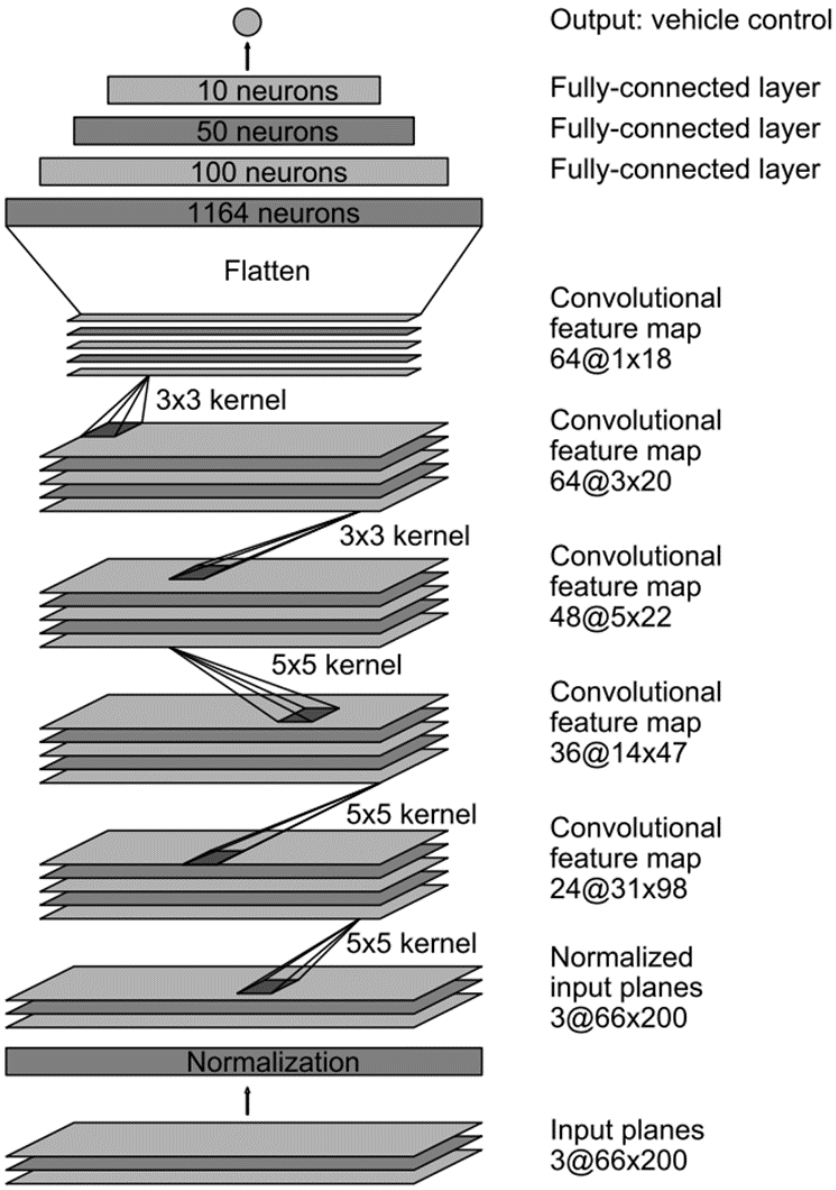


Fig: Sample Data

Dataset (Data Collection)

Data was collected with udacity self-driving car simulator. The Udacity simulator has two modes, training and autonomous, and two tracks. Training mode is to log the data for learning the driving behaviour. To do this I drove the car on track 1 keeping the car at centre of the lane and logged the data for 3 laps.
But one must understand what is meant to log the data here. So, let me explain what data is being logged (captured) and how.
Data capturing in a simulator is inspired by **Nvidia** paper, according to this paper, a car is mounted with 3 front cameras (one on right of bonet, one on centre and one at the left of bonet). Below image describes the setup of cameras and steering control (Image from Nvidia Paper).

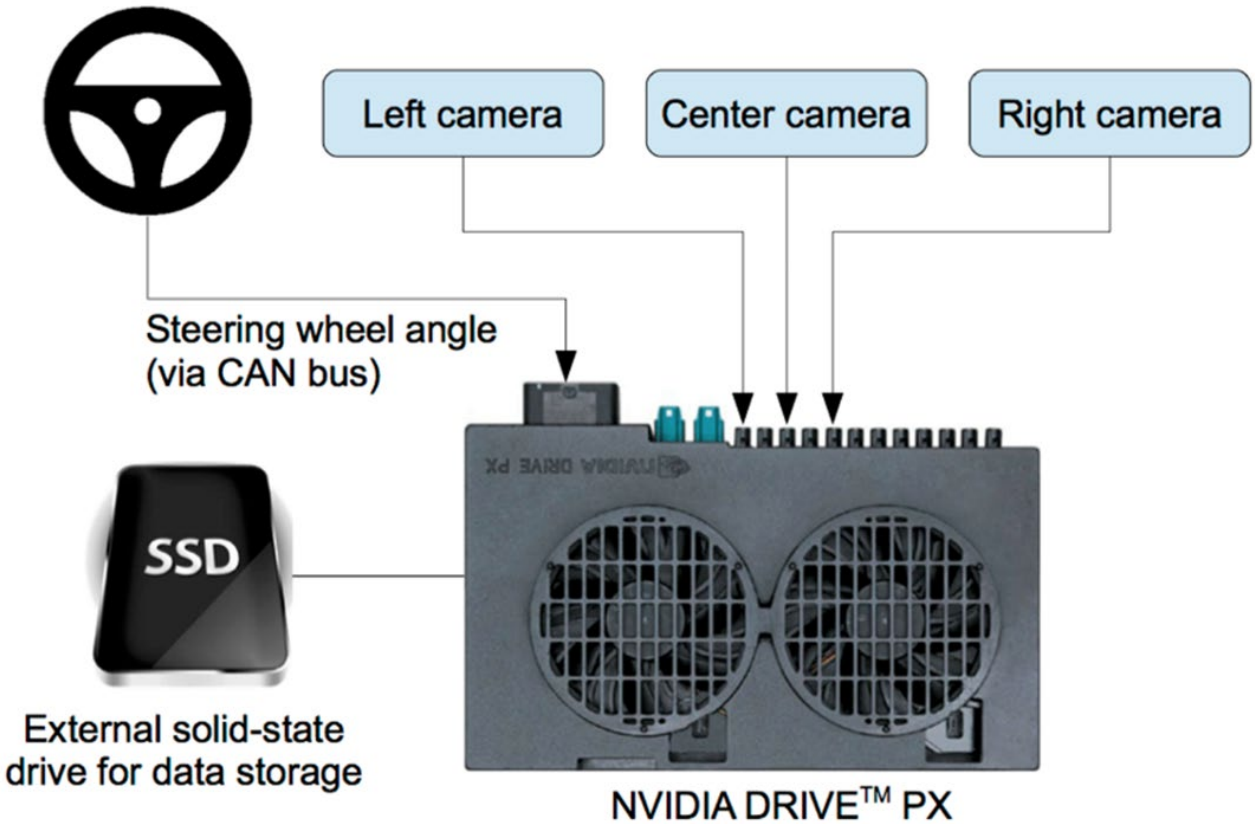


Fig: Camera Set-up

With above camera setup, udacity simulator collects three different images and corresponding steering angle and logs into a folder with csv file having each image location with the steering angle.

Understanding Data:

There are 3 cameras on the car which shows left, center and right images for each steering angle.

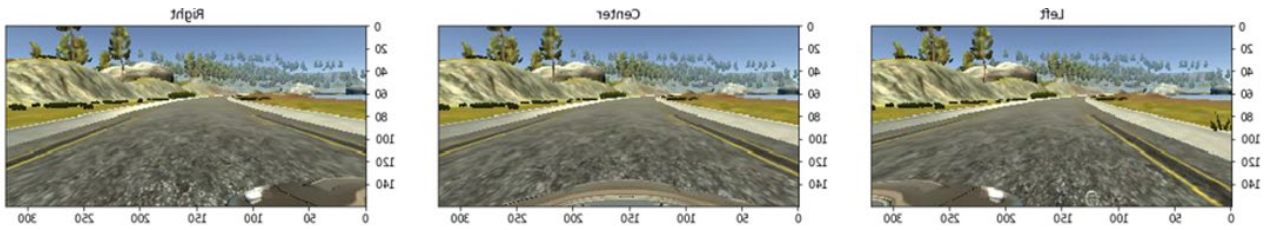


Fig: 3 camera images

Views plot

	center	left	right	steering	throttle	brake	speed
2	IMG/center_201	IMG/left_20	IMG/right_201	0	0	0	22.14829
3	IMG/center_201	IMG/left_20	IMG/right_201	0	0	0	21.87963
4	IMG/center_201	IMG/left_20	IMG/right_201	0	0	0	1.453011

After recording and save data, the simulator saves all the frame images in IMG folder and produces a driving_log.csv file which containts all the information needed for data preparation such as path to images folder, steering angle at each frame, throttle, brake and speed values.

Driving log

In this project, we only need to predict steering angle. So we will ignore throttle, brake and speed information.

Model Architecture:

The model has:

1. Two preprocessing layers, which I will describe later when talking about data.
2. Three convolutional layers with (5,5) kernels (24, 26 and 48 kernels per layer, correspondingly) with (2,2) strides, followed by

3. Two convolutional layers with (3,3) kernels (64 kernels per layer) with (1,1) strides, followed by
4. Three fully connected layers (with 100, 50 and 10 neurons, correspondingly), followed by
5. Output layer with one output neuron that controls the steering wheel.

We decided to use ELU (Exponential Linear Unit) activation, because there is evidence that it can be slightly better than RELU.

Layer (type)	Output Shape	Param #
cropping2d_1 (Cropping2D)	(None, 70, 320, 3)	0
lambda_1 (Lambda)	(None, 70, 320, 3)	0
conv2d_1 (Conv2D)	(None, 33, 158, 24)	1824
dropout_1 (Dropout)	(None, 33, 158, 24)	0
conv2d_2 (Conv2D)	(None, 15, 77, 36)	21636
dropout_2 (Dropout)	(None, 15, 77, 36)	0
conv2d_3 (Conv2D)	(None, 6, 37, 48)	43248
dropout_3 (Dropout)	(None, 6, 37, 48)	0
conv2d_4 (Conv2D)	(None, 4, 35, 64)	27712
dropout_4 (Dropout)	(None, 4, 35, 64)	0
conv2d_5 (Conv2D)	(None, 2, 33, 64)	36928
dropout_5 (Dropout)	(None, 2, 33, 64)	0
flatten_1 (Flatten)	(None, 4224)	0
dense_1 (Dense)	(None, 100)	422500
dropout_6 (Dropout)	(None, 100)	0
dense_2 (Dense)	(None, 50)	5050
dropout_7 (Dropout)	(None, 50)	0
dense_3 (Dense)	(None, 10)	510
dense_4 (Dense)	(None, 1)	11

Total params: 559,419
Trainable params: 559,419
Non-trainable params: 0

Fig: Model Architecture

Training and Validation:

In order to train a model, We used two generators — one for training and one for validation. Validation data generator was used to assess out-of-sample performance. Training generator was performing random data augmentation to improve generalization capabilities of the model, but validation generator was only performing preprocessing without doing any of the augmentation.
When training the model as described in NVIDIA paper, We noticed that training error quickly was becoming smaller than validation error, which is the sign of overfitting. To reduce that we introduced dropout layers after each convolutional and each dense layer with the exception of the output layer. After training the model with different values of dropout we stopped at 0.5 for the final model.

Data Pre-processing:

Pre-processing and Augmentation of images are required to enhance convergence, minimize overfitting and speed up the training process. The image processing pipeline was implemented in python using Computer Vision Library. The code for data processing, network model and training is available in the file *model.py*
The secret sauce in making the car drive itself well is not so much the architecture, but the data.
To achieve a more uniform distribution. We tried augment the data, also reduce the occurrence of low and zero angle data points, But the training time was too long and the car didn't drive smoothly.

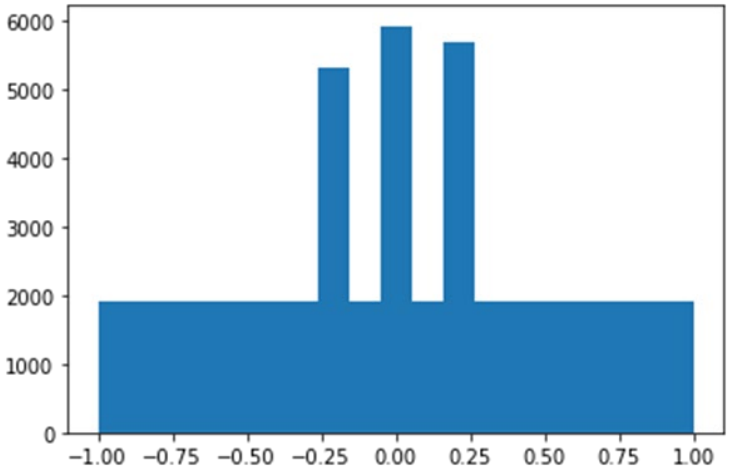


Fig: Pre-processed Data

So we didn't augment the data, but remove the data to make it more uniform distributed. Again, the car didn't drive very well due to dropped off too many data. (Here's the original distribution of the training data (the x-axis corresponds to steering angles and the y-axis is the data point count for that angle range; Blue bars represent overrepresented classes, removed during data pre-processing step.)

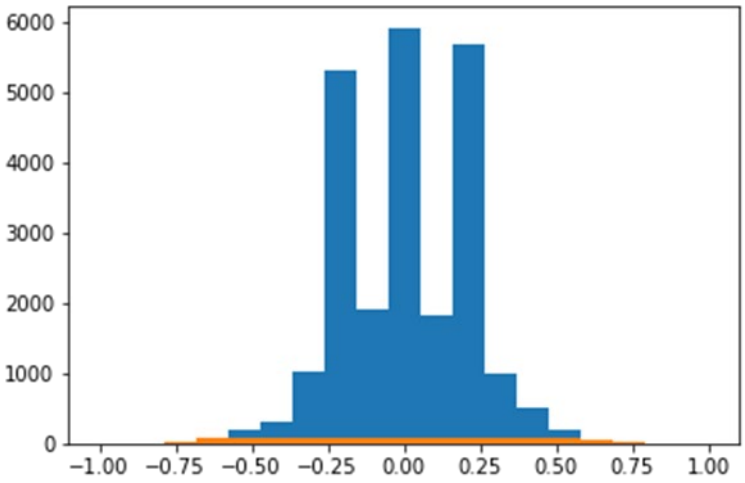


Fig: Original Data Distribution

After several try and error, we got a better result by not making the distribution uniformed overall, but make sure it's much closer to uniform for lower and zero turning angles.

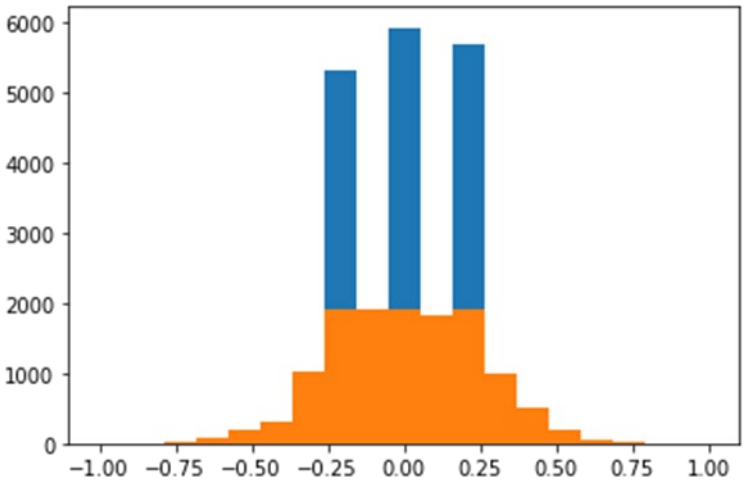


Fig: Improved Result

Optimizing the Model:

The model was optimized by implementing the following measures.

Initially the model was underfitting and the car showed a bias towards straight driving, failing to turn at curves. By adding recovery laps, more data samples using data augmentation discussed earlier, a drive on Track2, the bias towards straight drive was eliminated.

Later the car was taking wrong turns or driving along the edges of the track. I noted that the validation losses were significantly higher than the training losses. I identified that the problem was due to overfitting. This was minimized by adding dropout layers after all layers with drop probability at 0.4

To save the model with least validation loss, i used the ModelCheckPoint () function in Keras which saves the best model while continuing to run for 20 Epochs. I added an early stopping with the condition to stop if five consecutive epochs show no improvement to save time.

Challenges with the project:

- Data: As we are not good at playing video games, also driving skills are very bad. Therefore, it's impossible for us to get the good data from simulator. It was the most difficult challenge for me, getting a smooth driving data. Fortunately, Udacity provides the sample data.

- Training time: We used my local Linux server to train my model, but GPU was not fast enough. So we could only train the model once or twice per day, which is very frustrated

.

- Data pre-processing: Only very simple data pre-processing steps were taken. It might be beneficial to combine some of the steps that were implemented in the other projects to help, for example, highlight the lane lines more clearly so that the model can use less noisy input data.

Conclusion:

This project of Udacity is a challenging one specially for beginners in machine learning. This one involves collecting the training data by self. The task of collecting training data, devising a strategy for it can itself be daunting at first. Next comes the difficulty in setting up the model, processing the input data and optimizing the model parameters. More often we might have concluded that its impossible to solve. The project is based on deep learning, but it provides scope for us to learn deeply. That's the only way to solve this problem. Once we saw the car navigating the track with help of my trained model, the entire struggle to solve this problem vanished and we were filled with sheer joy.

References

- https://github.com/navoshta/behavioral-cloning
- https://github.com/naokishibuya/car-behavioral-cloning
- https://medium.com/@madhusudhan.d/guiding-self-driving-car-using-behavioral-cloning-9c24541a425d
- https://tina-chien-tw.medium.com/behavioral-cloning-project-3fadf2ae0db2
- https://towardsdatascience.com/behavioural-cloning-applied-to-self-driving-car-on-a-simulated-track-5365e1082230